Pack 03 · Inside the serving engine

Batching, scheduling and paged KV memory: how vLLM-style engines keep the GPU busy without breaking latency. Lessons 19, 26, 28, 30, 32–41, 44, 45. Answer each question out loud in about a minute, then check the card.

Q1 · Static vs continuous batching PRIORITY

Lessons: 28, 32, 33, 34

> **INTERVIEWER**
>
> A static batch of 4 slots gets replies of 50, 150, 400 and 800 tokens at about 5 ms per step. What share of the work is useful, when does the 50-token user get their answer, and when is static batching actually fine?

60-second answer

- **Useful share:** a static batch keeps running until its **longest** member finishes, so all 4 slots are held for 800 steps. That's a capacity of 4 × 800 = 3,200 slot-steps, but the real work is only 50 + 150 + 400 + 800 = 1,400 slot-steps, so the useful share is 1,400 ÷ 3,200 ≈ **44%**.
- **The 50-token user:** their reply is done after 50 steps, but they can't leave until the batch finishes at step 800. At 5 ms per step that's 800 × 5 ms = **4 s**, when they could have had their answer after 50 × 5 ms = 0.25 s.
- **The fix:** **iteration-level (continuous) batching** makes the decision after every step instead of once per batch. Finished requests leave right away and queued ones take their slots immediately, so no slot sits idle waiting for the longest reply.
- **When static is fine:** the useful share is roughly mean length ÷ max length, so static batching only hurts when lengths vary. When outputs are **uniform**, like a nightly job emitting one token per review, the mean equals the max and almost nothing is wasted.

Key numbers: static useful share = Σ lengths ÷ (slots × longest).

Likely follow-up: "Why is continuous batching the single biggest serving win?" Because it keeps the batch full at every step. Each step reads the weights once however many tokens it serves, so a full batch makes every weight read serve as many tokens as possible, and short requests no longer wait behind long ones.

> **TRAP**
>
> I didn't know what to divide by: capacity is slots × the longest reply, not the sum of replies.

Q2 · A ragged step: pad it or not? PRIORITY

Lessons: 33

> **INTERVIEWER**
>
> On one step, two requests decode one token each and a new request brings a 4-token prompt. A junior suggests padding all three to 4 tokens. What's wasted, and how does Orca's selective batching avoid padding?

60-second answer

- **The waste from padding:** padding all three requests to the longest length of 4 gives $3 \times 4 = 12$ slots, but only $1 + 1 + 4 = 6$ of them are real tokens. So half the slots are padding, which means **50% of the work is wasted**.
- **How selective batching avoids it:** most layers treat each token independently, namely the QKV and output projections, the MLP and the norms. So Orca **flattens all 6 tokens into one batch** for those token-wise layers, and no padding is needed because nothing in them depends on which request a token came from.
- **Why attention is the exception:** each token must attend only to **its own request's history** (its KV cache) under a causal mask. That's why **attention is split per request**, while everything around it runs on the flattened batch.

Key numbers: padded waste = 1 − real tokens ÷ (requests × longest).

Likely follow-up: "How does the attention kernel handle different lengths?" It reads each request's KV through that request's own block table (PagedAttention), so each request attends over exactly its own history and no padding is needed there either.

> **TRAP**
>
> I listed only the MLP as token-wise and didn't say why attention must be split.

Q3 · The scheduler is now on the clock PRIORITY

Lessons: 33

> **INTERVIEWER**
>
> With iteration-level scheduling, each GPU step takes 5 ms and the Python scheduler spends 2 ms per step deciding who runs, with the GPU idle meanwhile. What fraction is wasted, and why was this invisible before?

60-second answer

- **The wasted fraction:** each cycle is the 5 ms GPU step plus 2 ms of scheduling, so a cycle takes $5 + 2 = 7$ ms. The GPU sits idle for 2 of those 7 ms, so **$2 \div 7 \approx 29\%$** of GPU time is wasted and throughput drops to **$\approx 71\%$** of what it could be.
- **Why it was invisible before:** under request-level scheduling, the scheduler ran once per request, not once per step. A request lasts many steps, so the cost was spread thin and was negligible. Iteration-level scheduling runs it every step, which puts it directly on the clock.

- **The fixes:** the main one is to **overlap** the scheduling for step n + 1 with GPU step n (async scheduling), so the GPU doesn't wait for the CPU. Beyond that, make the scheduler itself faster and use CUDA graphs to cut launch time.

Key numbers: waste = CPU time ÷ (GPU time + CPU time).

Likely follow-up: "How would you spot it?" In a profiler trace you'd see GPU-idle gaps between kernels. You'd also see throughput that doesn't rise with batch size, since the GPU isn't what's holding it back.

> **TRAP**
>
> I divided 2 ÷ 5. The denominator is the whole cycle.

Q4 · Decode first, and sizing the token budget PRIORITY

Lessons: 34, 35, 45

> **INTERVIEWER**
>
> A teammate wants new prompts scheduled before running decodes "to cut TTFT". And for Llama 3.1 8B on an H100 (≈ 4.8 ms floor, then ≈ 16 μs per token), with 32 users decoding and a 20 ms inter-token target, what's the largest prefill chunk per step?

60-second answer

- **Why decodes go first:** if new prompts ran first, every user already streaming would **stall mid-answer** while the prompt is processed. Scheduling decodes first protects TPOT, and it also protects the KV cache already invested in the running requests.
- **How long a step takes:** a step has to read the weights, which takes ≈ 4.8 ms, and do the math, which costs ≈ 16 μs per token. The two overlap, so step time ≈ **max(4.8 ms, tokens × 16 μs)**, a max and not a sum.
- **The token budget:** to stay under the 20 ms target, a step can hold 20 ms ÷ 16 μs = 1,250 tokens. The 32 decoding users take 1 token each, which leaves 1,250 − 32 = 1,218 tokens, and rounding **down to a multiple of 128** gives **1,152 tokens** of prefill per step.
- **What this is called:** this is **chunked prefill**. Long prompts are split across several steps, so no single step blows the inter-token budget.

Key numbers: budget ≈ ITL target ÷ per-token cost; prompt room = budget − running decodes; steps for a prompt = ⌈prompt ÷ room⌉.

Likely follow-up: "A 6,000-token prompt with 150 decodes and a 25 ms target?" The budget is 25 ms ÷ 16 μs ≈ 1,560 tokens per step. The 150 decodes take 150 of those, which leaves 1,560 − 150 = 1,410 prompt tokens per step, so the prompt needs ⌈6,000 ÷ 1,410⌉ = 5 steps.

> **TRAP**
>
> I once added the math on top of 4.8 ms ("microseconds") instead of taking the max. Let units decide: ms ÷ μs gives a token count.

Q5 · Which gate is closed? PRIORITY

Lessons: 30, 35

> **INTERVIEWER**
>
> num_requests_running is stuck at exactly 64 for an hour, num_requests_waiting climbs past 200, KV cache usage is 40%, and steps are well under the latency target. What's blocking new requests?

60-second answer

- **The gates:** a waiting request is admitted only if it fits in **both** budgets: the number of sequences running (**max_num_seqs**) and the number of tokens this step (**max_num_batched_tokens**). It also needs free KV blocks.
- **Reading the symptoms:** KV usage is only 40%, so memory isn't the gate. Steps are well under the latency target, so the token budget isn't the gate either. Running is pinned at exactly 64 while the queue grows, which means the sequence cap is full: **max_num_seqs = 64**.
- **The fix:** raise max_num_seqs. Then re-check KV usage, preemptions and TPOT at the new level, since more running sequences use more KV and make each step heavier.

Key numbers: admitted only if running < max_num_seqs, tokens this step ≤ budget, and KV blocks are free.

Likely follow-up: "256 requests decoding, max_num_seqs 256, budget 8,192, and a 500-token prompt arrives?" Check each budget on its own. The tokens fit, because 256 decode tokens plus 500 prompt tokens is 756, well under 8,192. But 256 sequences are already running and the cap is 256, so the sequence budget is full and the new request waits in `num_requests_waiting` until one request finishes.

> **TRAP**
>
> Check each budget separately. I didn't know where to start on the 256-sequence case.

Q6 · Contiguous slabs vs paged blocks, and the block size PRIORITY

Lessons: 19, 28, 37, 38

> **INTERVIEWER**
>
> An engine reserves one contiguous 8,192-token KV slab per request. The pool is 50 GB and requests average 1,000 tokens (Llama 3.1 8B). How many fit, how much is wasted, what do paged blocks change, and why use 16-token blocks?

60-second answer

- **How many fit:** each token's KV takes 128 KiB, so one slab is 8,192 × 128 KiB = **1 GiB**, and a 50 GB pool holds ≈ **46 requests**. Each request only uses 1,000 of its 8,192 tokens, so each slab is just ≈ **12%** used. If we stored only the tokens actually used, the same pool would fit ≈ **380** requests.

- **Three kinds of waste:** **reserved** space is held for tokens that haven't been generated yet. **Internal fragmentation** is the rest of the slab that's never used because the request stops short. **External fragmentation** is free gaps that are too small to hold a new slab. You can't just compact, because that would copy gigabytes of live KV and stall everyone.
- **What paged KV changes:** KV is stored in fixed-size blocks that are handed out as tokens arrive, and each request has a block table that tracks its blocks. Only a request's last block can be partly empty, so the waste is **at most one partial block per request**.
- **Why 16-token blocks:** with 1-token blocks the tables get huge and reads are scattered. With 256-token blocks a request can waste up to 255 tokens. **16** sits in between, keeping waste around 8 tokens per request while reads stay efficient.

Key numbers: blocks = ⌈tokens ÷ 16⌉; token t lives in table[t // 16], slot t % 16.

Likely follow-up: "What's the catch of paging?" The engine admits requests based on their current size, not their final size. So if many of them grow at once, the pool runs dry mid-generation and the engine has to preempt some of them.

> **TRAP**
>
> I dropped the 2 bytes for BF16 and divided upside down; a request count must come out bigger than 1. And small blocks cost bookkeeping and scattered reads, not "recomputation".

Q7 · "PagedAttention must approximate, and its kernel is slower"

PRIORITY

Lessons: 39, 40

> **INTERVIEWER**
>
> An interviewer says PagedAttention reads scattered keys and values, so it must approximate attention, and its kernel is 20–26% slower. How can vLLM claim 2–4× higher throughput? And what does paging do for n = 4 samples of one 1,000-token prompt?

60-second answer

- **It's exact:** PagedAttention does the same math as normal attention; it just reads K and V through a block table. The 20–26% kernel slowdown comes from that indirection inside the kernel, not from any approximation.
- **Why the server still gets faster:** paging wastes almost no memory, so **far more requests fit** at once. Decode is memory-bound, so a bigger batch shares each weight read across more tokens. Attention is only a small part of the step, so a slower attention kernel matters far less than the much bigger batch, and the server gets much faster overall.
- **Sharing the prompt:** the 1,000-token prompt needs ⌈1,000 ÷ 16⌉ = **63 blocks**, and all four samples can point at the same 63 blocks. Four separate copies would need 4 × 63 = 252 blocks.
- **Copy-on-write:** the last prompt block is only partly full (1,000 = 62 × 16 + 8), so each sample's first new token lands in a shared block, which triggers **copy-on-write**. The first three writers each get their

own copy, and by then the last writer is the only one left on the original, so it can write in place. That's 3 copies, so 63 + 3 = **66 blocks** in total.

Key numbers: copies for n samples sharing a partial block = n − 1.

Likely follow-up: "Where else does block sharing pay off?" In prefix caching, where different requests share the blocks of a common prefix (pack 05), and in beam search.

> **TRAP**
>
> I blamed the kernel slowdown on allocation and eviction. Those are scheduler work; the kernel cost is block-table indirection.

Q8 · Preemption: who goes, how, and when to worry PRIORITY

Lessons: 36, 40, 41

> **INTERVIEWER**
>
> num_preemptions_total climbs every minute and p99 latency spikes; requests average about 2,000 tokens. Who gets evicted and how? A teammate suggests evicting the biggest request instead. And is a KV cache pinned at 100% always a problem?

60-second answer

- **Size of one request:** at 128 KiB per token, a 2,000-token request holds 2,000 × 128 KiB ≈ **262 MB** of KV, which is what each preemption frees.
- **Who goes:** evict the **newest** request, because it has made the least progress, so the least work is lost. Evict it **whole**, since attention needs the full history and part of a KV cache is useless. Evicting the biggest request instead is worse, because that one has done the most work and you'd throw all of it away.
- **How it's evicted:** there are two ways. **Recompute** drops the KV and re-runs prefill later, which costs compute, while **swap** moves the KV to CPU memory, which costs PCIe transfer time. Neither fixes the cause, so the real fix is to admit fewer requests (max_num_seqs) or add KV memory (fp8 KV, more GPUs).
- **Is 100% KV usage a problem:** not on its own. **100% usage with zero preemptions is healthy**, because freed blocks stay cached for reuse. The real pressure signal is **rising preemptions**, which users feel as stalls in the middle of a stream.

Key numbers: steps until the pool runs dry = free blocks × 16 ÷ running requests (2,000 × 16 ÷ 250 = 128 steps ≈ 2.6 s at 20 ms).

Likely follow-up: "Why didn't the scheduler see it coming?" Because it admits each request based on its current size, not its final size. Requests keep growing as they decode, so memory that looked free at admission runs out later.

> **TRAP**
>
> I answered with queue order (who starts) instead of eviction (who leaves), and called queue length the KV-pressure signal. It's preemptions.

Q9 · Cancellation and fairness across customers PRIORITY

Lessons: 36

> **INTERVIEWER**
>
> A user closes the tab at token 300 of a reply capped at 1,000 tokens and the server never notices. What does it cost? Separately, client A sends 90% of traffic and small client B has terrible p99 TTFT. The PM proposes a hard per-client rate limit. What would you do instead?

60-second answer

- **Cost of not cancelling:** the user left at token 300 but the reply is capped at 1,000, so the server generates 1,000 − 300 = **700 tokens** for nobody. The request also keeps growing to 1,000 × 128 KiB ≈ **131 MB** of KV that no one will read. Aborting on disconnect frees its slot and its KV at once.
- **Why not a hard rate limit:** a hard per-client limit **rejects work and idles the GPU**, even when nobody else is waiting to use it.
- **What to do instead:** use a **virtual token counter (VTC)**. It counts each client's weighted served tokens and admits next from the client with the lowest count, so small client B gets in ahead of A. It **reorders admission and never rejects** anything.
- **Why it's better:** VTC is work-conserving, which means the GPU never idles while there is work to do. So when B goes quiet, A gets the whole GPU.

Key numbers: wasted tokens = max_tokens − tokens when the user left.

Likely follow-up: "What stops B from saving up credit while idle?" When B comes back, its counter is lifted to the current minimum. So B can't build up a low count while away and then burst ahead of everyone.

> **TRAP**
>
> I called VTC "rate limiting". It never rejects anything. And on cancellation I described the fix instead of computing the waste asked for.

Q10 · Why RAG traffic slows the chat users PRIORITY

Lessons: 26, 44, 45

> **INTERVIEWER**
>
> Chat (500-token prompts) and RAG (6,000-token prompts) share one H100. Since RAG was added, chat streams more slowly although chat traffic is unchanged, yet p99 inter-token latency looks the same as before. Explain both.

60-second answer

- **Prefill interference:** each long RAG prefill takes up a whole step, $\approx 6{,}000 \times 16\ \mu s \approx 96$ ms for a 6,000-token prompt. Every chat stream that's already decoding has to wait for that step, so chat users see their stream pause even though chat traffic hasn't changed.
- **Memory:** RAG requests carry big KV caches, and every decode step is shared by all running requests. So each shared step reads more bytes, and every chat token takes longer.
- **Why p99 looks the same:** take lesson 44's test: 32 chat streams for 40 s, with one long prompt injected every 10 s (at 5, 15, 25 and 35 s). That's 4 long prefills, so each user sees **4 stalls**. With $\approx$ 10 ms decode steps, each user also sees about 40 s $\div$ 10 ms $\approx$ 4,000 token gaps ($\approx$ 3,940 once the 4 long steps take their share of the 40 s). p99 ignores the slowest 1% of gaps, and 4 $\div$ 3,940 $\approx$ 0.1% is below that 1%, so **p99 is blind** to them. It would only move if stalls were more than 1% of gaps, for example one long prompt every second. To see them, report the max, p99.9, a stall count or the gap histogram.
- **The fix:** use **chunked prefill** (Q4), so a long prompt is split across steps and never freezes the chat streams for a whole long step. The other option is to separate prefill and decode onto different GPUs (disaggregation).

Key numbers: a percentile p only sees events more frequent than $1 - p$.

Likely follow-up: "Why not just look at TPOT?" TPOT is an average of the gaps, so a few long freezes get averaged in with thousands of normal gaps and vanish.

> **TRAP**
>
> I said chat requests queue. They're already decoding and get paused by prefill. I also forgot the memory reason.

Build it yourself

- **`exercises/18_19_paged_kv.py`**: simulates reserving the max length versus paged blocks with preemption. It shows paging runs 4.5× more requests at once at the cost of evictions. Run `uv run exercises/18_19_paged_kv.py`.
- **`exercises/32_36_batching_sim.py`**: a static versus continuous batching simulator (steps, useful share, average finish time). Run `uv run exercises/32_36_batching_sim.py`.
- **`exercises/37_41_block_manager.py`**: a block manager with a free queue, reference counts and copy-on-write (`allocate`, `fork`, `append`, `free`). Run `uv run exercises/37_41_block_manager.py`.